Exact Evaluation of Adaptive Prefix-Cache Continuations

Mohammad Ali Seyed Mahmoud
Georgia Institute of Technology
m.seyedmahmoud@gatech.edu

Technical report, v3.1. November 12, 2026. · This paper is licensed under CC BY 4.0.

Abstract

Tied request orders can leave different cache states. Under ordinary LRU with prefix paths charged at entry, we show how to score a known, finite continuation from those states even when its request order depends on cache hits. Equivalent starting states can share a score; a sufficient persistence test sometimes gives the score without replay. On one previously examined Mooncake tool/agent panel (128 blocks, decision output), the two shortcuts reduced historical replays from 267,672 to 46,610. Our original implementation and our adaptation of focused LRU saved exactly the same replays in all eight conditions. The original used more mean native CPU in six. On the highlighted panel, the focused implementation took 66.59 microseconds per decision versus 89.00 microseconds for the original; the local caller's wall time fell 16.9%. The historical policy saved 0.204% of pooled modeled prefill work over exact immediate ordering on Mooncake and zero on two external episodes. The evidence supports a specialization of focused LRU and a measurement of evaluator cost. We have not established an end-to-end benefit for an independent planner.

1 Introduction

Suppose two orders for the same ready requests incur the same prefill work and leave the same prefixes resident, but leave those prefixes in different recency orders. The next admission can evict a different prefix. A scheduler that prices the two exits by simulation must account for a second dependence: a continuation that prioritizes cache hits can issue its requests in different orders from those exits. What computation can the two simulations safely share?

We give an exact method for scoring a finite continuation from multiple cache states and measure its computational cost. For each continuation, the evaluator identifies a sufficient cache observation, tests whether initially resident prefixes survive and initially absent ones miss exactly once, and computes each remaining distinct score once when required. It can propagate a complete cache or only the relevant focused state during residual replay. The construction preserves candidate scores and the chosen full exit. Whether historical demand predicts the next real requests is a separate question answered by trace replay.

2 Related Work

Mattson et al. use LRU stack positions to evaluate a scalar address sequence across cache capacities. Touzeau et al. classify cache hits and misses along program control flow by tracking the set of addresses younger than a focal address. Stock et al. study persistence, the property that all accesses to a block within a scope incur at most one miss in total. These results provide a basis for analyzing prefix paths admitted together. Givan et al. give the general principle for grouping states that preserve action-dependent transitions and rewards [1, 2, 3, 4].

Dexter et al. introduced k-LPM for scheduling requests with prefix reuse and latency constraints [5]. The historical continuations here use an adaptation of that rule. Their service order can therefore depend on which prefixes are resident at each step.

3 System Model and Problem Formulation

A node identifies a **complete prefix**. Equal content blocks under different parents identify different nodes. A reachable state F is a prefix-closed list of at most C distinct nodes, in most-recent-first retention order. The cacheable part of a request is a shallow-to-deep path $P = (p_1, \dots, p_l)$ of distinct nodes, capped so $l \le C$. Hits are observed at entry. The path is then admitted under a single service stamp and protected during that admission. Its nodes are placed shallowest first, old copies are removed, and the remainder retains its order. Thus

$$c_P(F) = l - |P \cap F|, \qquad T_P(F) = \mathrm{take}_C(P \parallel (F \setminus P)) \,. \tag{1}$$

Set deletion in (1) preserves list order. A reachable ancestor has been touched at least as recently as its descendants. The depth rule places ancestors ahead of descendants with the same stamp. Truncation from the end therefore removes eligible leaves, and the cap on l protects the entire admitted path. For an input of L positions and block width b, the actual prefill charge is $(L - bl) + b\, c_P(F)$. The first term covers all positions outside the reusable path, including the compulsory final token. Its sum is constant across orders serving the same requests.

The charge convention changes the answer even when a scalar simulation produces the correct exit. At capacity two, start at (b, a) and request (a, x), where x is a child of a and b is another root. At entry a hits and x misses. Scalar touches x then a also leave (a, x), but the first touch evicts a before the second, giving two misses. An exact continuation evaluator must preserve both the transition and the entry-time charge.

For a legal order a of the current cohort, write w_a for its immediate missing-block count and F_a for its complete exit state. The evaluated policy first restricts attention to $\mathcal{A}_0 = \arg\min_a w_a$. From each eligible exit it evaluates the same finite list of historical continuations. In the implemented policy, the current cohort contains up to four requests. The last sixteen completed cohorts supply up to eight disjoint chronological pairs $\mathcal{H}_j$, formed from the most recent end. Each pair starts afresh at F_a. With at least one pair available, the policy selects an order minimizing

$$\sum_{j=1}^{m} J(F_a, \mathcal{H}_j), \qquad a \in \mathcal{A}_0,$$

where J counts missing blocks under the specified continuation rule. Ties retain the deterministic immediate incumbent when eligible among the minimizers, then favor arrival-lexicographic order. An empty history of complete pairs returns the incumbent.

4 Exact Continuation Evaluation

4.1 Cache-State Equivalence

Let U cover every node that **any permitted branch** of a specified finite continuation can access. For each resident $x \in U$, $\Pi_U(F)$ records its identity and absolute, one-based rank $r_F(x)$. Unlisted labels in U are absent. The gaps in these ranks retain the number of intervening outsiders even when their identities are discarded.

Proposition 1. Fix the capacity, namespace, pending workload and metadata. Suppose the selector, its internal updates, and its stopping rule depend on cache state only through observations

recoverable from Π_U, including the charge for a chosen request. For equally initialized controllers, equality of the initial keys implies identical actions, observed charges and total continuation work. For randomized controllers the statement holds pathwise under an identical random stream.

Proof. For any available path $P \subseteq U$, the key identifies its resident nodes. Prefix closure makes them an initial segment, giving the hit length and c_P. After admission, p_i has rank i. For an initially resident $x \in U \setminus P$ of rank r, the prospective new rank is

$$\tilde{r} = l + r - |\{p \in P : r_F(p) \leq r\}|, \tag{2}$$

using infinity for absence. The whole path moves ahead of x, and old path entries already ahead of x are removed once. The node survives exactly when $\tilde{r} \leq C$. An absent, untouched tracked node stays absent. Thus each permitted action determines the same charge and successor key from either initial state. Equal observations give the same controller decision and internal update; induction over the finite continuation proves the result. The archived k-LPM selector satisfies the restriction: it alternates oldest pending request with longest currently cached prefix, uses arrival order to resolve ties, and restarts the alternation at each cohort.

Absolute ranks matter. With capacity two, serving singleton roots c, a from (a, b) costs one miss, and from (b, a) costs two. In both starts a is the sole resident label in the continuation support. Conversely, if the continuation consists only of a, these different ranks give the same zero cost. Equality of keys is sufficient; necessity and global minimality do not follow. The historical application knows U from completed requests. Using only the labels seen along one realized branch of a more general adaptive procedure could omit a possible access.

4.2 Cache Persistence

Proposition 2. Now require U to be the exact union of labels that will **all be touched** in the continuation. Define $u = |U|$, $h = |F \cap U|$, and

$$z = \begin{cases} \max_{x \in F \cap U} r_F(x) - h, & h > 0, \\ 0, & h = 0. \end{cases}$$

If $u + z \leq C$, the exact missing-block count is $u - h$ for every permitted service order.

Proof. The deepest initially resident tracked label has $h - 1$ tracked labels ahead of it. Its number of younger outsiders is therefore z, an upper bound for every initially resident tracked label. Until such a label is first touched, admission moves only members of U forward. Before each truncation there can be at most z outsiders and $u - 1$ other tracked labels ahead of it. It survives, including at equality with capacity. Once touched, a tracked label can have only other tracked labels ahead of it and likewise survives. Every initially absent label incurs one miss on first use, and all repeated uses hit. This reasoning includes partially filled caches, $h = 0$ and empty paths. For an empty continuation the charge is zero. Apply the formula to capped paths and add compulsory positions separately via (1).

This is a sufficient persistence certificate. Stock, Hahn and Reineke call a block persistent over a scope when its accesses together incur at most one cache miss. The proof here specializes the standard LRU stack-distance argument to a known starting cache and entry-charged, whole-path admissions. It certifies zero misses for the initially resident labels and exactly one for each initially absent label, yielding the total. It does not decide persistence whenever the bound fails [3, 1].

The coverage distinction is essential. A superset of possible accesses suffices for Proposition 1. An unvisited, initially absent label in that superset would be counted spuriously by Proposition 2. The frozen historical pairs serve every request. When the sufficient test fails, the evaluator can still recover the cost by exact replay or by reusing an equal projected score.

4.3 Score Reuse and Order Selection

First group immediately optimal orders that reach the same **complete** exit, keeping the representative preferred by the dispatch tie rule. For each historical pair, the key in Proposition 1 induces a further partition of those exits. Evaluate certified costs directly and reuse each unresolved class's simulated cost across its members. Add the score to each candidate's own sum. Partitions are pair-specific: two candidates can share a term for one pair and have different terms for another. If every candidate has the same term, decision-only evaluation can skip it. Absolute scoring computes that term once and restores it for every candidate.

These operations preserve the ordering of objective values, including ties. The evaluator commits the winning order's **own complete exit**. A simple example shows why: serving only a from (x) or (y) at capacity two costs one miss and gives the same terminal key on $\{a\}$. The resulting complete caches (a, x) and (a, y) differ on the next real request for x. Only a scalar historical term was shared.

5 Implementation

For each resident node x, let Y_x denote the set of nodes ahead of it in LRU order. Absence has its own marker. At admission, the implementation charges all path misses before updating recency. Each p_i acquires the preceding $i-1$ path labels as its younger set. An untouched resident x has prospective younger set $Y_x \cup P$ and survives precisely when its size is less than C. All these sets refer to the same candidate execution. Write $A_x = Y_x \cap U$ and $b_x = |Y_x \setminus U|$. On reachable states, $r_F(x) = 1 + |A_x| + b_x$. Conversely, the resident labeled ranks identify A_x as the tracked residents ahead of x and give $b_x = r_F(x) - 1 - |A_x|$. Absence markers agree. Since no outsider is touched, the atomic update for an untouched resident becomes $(A_x \cup P, b_x)$ before truncation, exactly the update in (2). This establishes mutual reconstruction for the **correlated** outsider-anonymized observations from one cache. Independently combining possibilities for different focal labels could produce an impossible joint order.

Both implementations use the persistence condition in Proposition 2 and share scores using the same projected keys. The comparison measures the cost of propagating the complete cache versus the focused state when replay is still necessary.

Evaluator	Survival test	Score sharing	State on residual replay
FULL = RANK-00	Off	Off	Complete candidate exit
RANK-10	On	Off	Complete candidate exit
RANK-01	Off	On	Complete representative exit
RANK-11	On	On	Complete representative exit
FOCUSED	On	On	Tracked identities and their absolute ranks

FOCUSED is our adaptation of focused-LRU principles, not the published POPL analyzer. It uses the same candidate generator, certificate, memo and tie rules as RANK-11. Only when replay is needed does it scan the representative and initialize the reduced state. It then obtains each adaptive choice from the current tracked hits and advances the ranks after each atomic admission. RANK-11 instead runs the same continuation on the complete witness. FULL and RANK-00 share one code path because the witness-based implementation with both shortcuts disabled has no use for a projected key. The benchmark compares our two implementations; it supplies no runtime comparison with Touzeau et al.'s software.

Computational complexity. For a historical pair with E distinct eligible exits, full evaluation can require E replays. With both shortcuts, it needs one replay per distinct uncertified class whose cost is required. Candidate generation is common work. The implementation scans up to EC slots to

identify varying ranks, charges certificate and key construction, and stores class scores in a pair-local linear memo. If L exits require key lookup and they form D unresolved classes, key comparisons alone can cost $O(LDC)$. When every exit is distinguishable and uncertified, classification adds work without saving replays. The focused kernel trades a scan to convert each residual witness for updates over tracked residents. Its two buffers reserve space at the compiled capacity. The history index also records identities encountered over the episode, so a sixteen-cohort history does not bound its storage.

6 Experimental Evaluation

6.1 Experimental Setup

The matched experiment reused 56 Mooncake bins [9] after their scheduling outcomes were known: 28 per traffic type, with 1,500 conversation cohorts and 2,745 tool/agent cohorts per capacity. All methods received the same arrivals, bin resets, warm-up, 512-token blocks and current-order search. Each traffic, capacity, output and implementation combination had three fresh-process repetitions on one Apple M2. The repeats measure timing variation on identical inputs; they supply no independent workloads, held-out timing validation or cross-machine estimate. Absolute output includes the integer historical sum and preferred order for every distinct eligible full exit. Decision output contains the chosen order and complete exit.

6.2 Correctness

Synthetic verification covered 3,287 decision inputs, 9,304 literal focused updates and 4,000 checked native continuation trajectories. For two integration bins at both capacities, all-candidate scoring was independently checked against the stamped-leaf oracle. Across integration and measured bins, 8,766 cohort–capacity cases supplied 90,954 reference candidate scores. Every absolute timing cell reproduced the expected score vectors, and every cell reproduced the selected orders, committed complete exits and episode prefill. These finite checks test the implementation of the propositions.

6.3 Runtime and Replay Counts

Table 1: Mean complete native CPU time (μs per measured cohort). Each method receives identical requests within an output interface. Includes parsing, candidate construction, historical evaluation and state maintenance. FULL = RANK-00.

Output	Traffic	Blocks	FULL	RANK-10	RANK-01	RANK-11	FOCUSED
Decision	Conversation	128	269.94	172.05	29.85	29.94	30.02
Decision	Conversation	32	57.00	55.70	14.05	13.77	14.28
Decision	Tool/agent	128	281.63	146.02	156.37	89.00	66.59
Decision	Tool/agent	32	54.79	53.29	40.30	39.72	33.69
Absolute	Conversation	128	273.19	174.75	51.04	42.50	37.65
Absolute	Conversation	32	57.60	57.32	20.57	20.48	18.54
Absolute	Tool/agent	128	279.21	145.53	157.24	90.07	67.78
Absolute	Tool/agent	32	56.32	54.77	42.58	42.27	35.89

Replay counts. In the 128-block tool/agent decision cell, FULL performed 267,672 pair-by-exit replays, certification alone 114,705 and sharing alone 118,093. Both strong implementations performed 46,610 (82.6% fewer than FULL in this cell). The counters reconcile cohort by cohort: 2,648 common pair terms were skipped before certification; weighting each by its number of eligible full exits accounts

for 26,391 evaluations. The remaining 241,281 checks yielded 139,871 direct scores; of the other 101,410, memo reuse supplied 54,800 and replay supplied 46,610. Adding certification to sharing eliminated 71,483 replays; adding sharing to certification eliminated 68,095. Their standalone reductions overlap by 81,484. RANK-11 eliminated **zero** additional replays relative to FOCUSED in any of the eight cells and used more mean complete native CPU in six.

CPU and wall time. In that same cell, FULL to FOCUSED reduced mean complete native CPU by 76.4% (281.63 to 66.59 μs). At 32 blocks the corresponding tool/agent reductions were 38.5% for decisions and 36.3% for absolute scores. Comparing the two strong implementations at 128 blocks, the historical phase, including checks and sharing, took 38.42 μs for FOCUSED and 60.10 μs for RANK-11. Native work outside that phase accounted for 28.17 and 28.90 μs respectively. Complete native CPU fell 25.2%, while caller wall, including IPC and response decoding, fell 16.9% (133.35 to 110.80 μs). These clocks do not measure serving latency.

Effect of workload. For conversation decisions at 32 blocks, adding certification to sharing saved zero replays (151 either way). At 128 it reduced 223 to 156, with no mean CPU gain: RANK-01 took 29.85 μs and RANK-11 took 29.94 μs. RANK-11 and FOCUSED both took about 30 μs there and about 14 μs at 32 blocks. For absolute scores on conversation at 128 blocks, the common terms still have to be valued: adding certification to sharing reduced replays from 12,182 to 6,975 and complete native CPU from 51.04 to 42.50 μs (RANK-01 to RANK-11).

Comparison with the pooled benchmark. The preceding experiment pooled the 4,245 measured Mooncake cohorts across both traffic types at 128 blocks. Mean native evaluator CPU fell from 283.03 μs with full-state replay without the combined shortcuts to 68.72 μs with the optimized tie-only evaluator, a **4.12× evaluator speedup**. Its corresponding local caller-wall speedup was 2.93×. Table 1 gives the subsequent matched, traffic-specific comparison with FOCUSED. Neither experiment timed an end-to-end serving system.

6.4 Scheduling Results

Exact evaluation guarantees the registered policy's choice. A selection that minimizes immediate work from one entry state can still increase complete-episode work. In the earlier immediate-order study with queue-aware eviction, the root bound certified 2,452 of 2,453 measured cohorts. In the remaining case the chosen order saved 512 positions immediately and incurred 2,048 additional positions two cohorts later. The result motivated pricing the exit a decision leaves behind.

In the subsequent ordinary-LRU study, the 56 Mooncake bins were a census of the then-unexposed bins remaining in the two traces. At 128 blocks, the historical tie-breaker saved 244,736 modeled prefill positions over exact immediate ordering across 15,432 measured requests: **478 blocks of 512 positions, or 0.204%** of the comparator's 119,746,315 positions. All savings were on tool/agent traffic; conversation tied. The different planner permitted to sacrifice immediate work saved 249,344 positions pooled. Its separately registered mean saving was 12.23 positions per request with requests averaged within bins, then bins and traffic types weighted equally.

The external Qwen/Bailian study [10] used one continuous episode per workload, each with 1,000 FCFS warm-up requests followed by 20,000 measured requests. In the primary 512-token, 128-block chat condition, history-informed and exact immediate ordering both consumed **43,320,479 positions**. Although 4,643 of 5,200 cohorts offered multiple immediately optimal exits, history retained the incumbent in every primary chat cohort. Two of three frozen random tie controls obtained lower episode work, demonstrating that another exit choice could matter on that episode. On API traffic both policies consumed **17,516,341 positions**. All selected requests had distinct chat roots. The release lacked tenant identities, and the frozen lineage-isolation rule precluded cross-request hits in that episode. The API episode thus measures overhead under lineage isolation. At both resolutions and capacities, neither external workload saved prefill work over immediate ordering.

A separate CPU replay of 3,000 held-out FreeInference requests allowed each request to start prefill up to 500 ms later than under first-come, first-served (FCFS). Across 36 primary configurations, the best tested order with LRU eviction saved at most 1.323% of modeled prefill work relative to FCFS. None reached the prespecified 3% threshold through ordering alone. Nine configurations met the criterion when the offline reference could also change eviction. The full ordering pilot on GitHub gives the protocol, per-configuration results and independent audit.

7 Discussion and Limitations

Mooncake reset the cache by bin and warmed it over each bin's first quarter. Qwen/Bailian used continuous measured state and reconstructed prefix identities within documented chat lineages. Source, namespace, warm-up and episode boundaries changed together. Mean intentional cohort collection waits on external chat and API were 272.4 and 68.2 ms per request. Those waits are separate from local scheduling time. At native resolution on continuous chat, the optimized index ended with 1,620,275 slots including its sentinel, with only 5,696 identities active in the retained history. The experiments measure modeled prefill and local CPU, wall time and memory. Live accelerator latency, production throughput, energy and monetary benefit remain unmeasured.

The historical scheduler consumes only the winning action and its full exit. Our verification uses the complete vector of scores. For a decision-only caller the fair comparator is the fastest method that returns the same action, including the deterministic tie and full exit. Bounds or early termination are eligible even if they never materialize losing scores. The experiment did not benchmark such a method. The measured microseconds per cohort also do not remove the external cohort collection waits of 68–272 ms per request. We screened three independent planners. WAKMan replays recent traffic to choose an eviction policy. Optimal Prefix Hit Recursion scores the leading fields shared by consecutive rows in a proposed table layout. KVSET obtains hit rates at different capacities from one LRU history. None of the three requires exact continuation costs from alternative cache states at a fixed capacity. The screen found no qualifying planner to profile [6, 7, 8].

8 Conclusion

The demonstrated contribution is a specialization for exact, multi-exit evaluation: entry-charged paths, known finite continuation support, and a selector whose cache observations are preserved. It measures how certification and sharing overlap, and where reduced-state propagation lowers local CPU. It provides limited incremental novelty over focused LRU and no demonstrated deployment benefit.

Artifact Availability

The eight timing cells, evaluator implementation, verification code, trace sources and build instructions are available in the public GitHub artifact (revision `7a2f9a970a`). Repository: https://github.com/smahmoud66/prefix-cache-studies.

Tool use. OpenAI Codex assisted with preparing the manuscript and computational artifact and with running the release checks. The reported measurements come from the archived experiments.

References

[1] Mattson, Gecsei, Slutz and Traiger. Evaluation Techniques for Storage Hierarchies. IBM Systems Journal, 1970.

[2] Touzeau, Maïza, Monniaux and Reineke. Fast and Exact Analysis for LRU Caches. POPL, 2019.

[3] Stock, Hahn and Reineke. Cache Persistence Analysis: Finally Exact. RTSS, 2019.

[4] Givan, Dean and Greig. Equivalence Notions and Model Minimization in Markov Decision Processes. Artificial Intelligence, 2003.

[5] Dexter et al. LLM Query Scheduling with Prefix Reuse and Latency Constraints. NeurIPS, 2025.

[6] Morisetty et al. WAKMan: Workload Aware KVCache Manager. arXiv:2610.36355, 2026.

[7] Liu et al. Optimizing LLM Queries in Relational Data Analytics Workloads. MLSys, 2025.

[8] Li et al. The KV Cache Working Set: Online Capacity Planning for LLM Inference Systems. arXiv:2609.27746, 2026.

[9] Mooncake trace authors. Mooncake FAST 2025 release: production conversation and tool/agent traces. Dataset, revision 3cca71d.

[10] Qwen/Bailian trace authors. Qwen/Bailian anonymized usage traces. Dataset, revision 5f7439c.